

```

const TelegramBot = require("node-telegram-bot-api");
const fetch = require("node-fetch");
const fs = require("fs");

// — CONFIG —————
const TELEGRAM_TOKEN = process.env.TELEGRAM_TOKEN;
const TOMTOM_KEY = process.env.TOMTOM_KEY;
const APIFY_TOKEN = process.env.APIFY_TOKEN;
const RAPIDAPI_KEY = process.env.RAPIDAPI_KEY; // AeroDataBox
const FOOTBALL_KEY = process.env.FOOTBALL_KEY; // football-data.org
const TICKETMASTER_KEY = process.env.TICKETMASTER_KEY; // ticketmaster
const CHAT_ID = process.env.CHAT_ID;

const TAX_RATE = 0.16;
const KAWA_SHARE = 0.40;
const FIXED_COMPANY = 1000;
const FIXED_INSURANCE = 500;

const DATA_FILE = "/tmp/finance.json";
const HOME_BASE = { name:"Herne", lat:51.5393, lon:7.2261 };

const bot = new TelegramBot(TELEGRAM_TOKEN, { polling: true });

// — DATA STORAGE —————
function loadData() {
  let d;
  try { d = JSON.parse(fs.readFileSync(DATA_FILE, "utf8")); }
  catch { d = {}; }
  // Backfill missing keys (handles old files from older versions)
  if (!Array.isArray(d.salam)) d.salam = [];
  if (!Array.isArray(d.kawa)) d.kawa = [];
  if (!Array.isArray(d.expenses)) d.expenses = [];
  if (!Array.isArray(d.strikes)) d.strikes = [];
  if (!d.alertedFlights || typeof d.alertedFlights !== "object") d.alertedFlights
  if (!d.alertedMatches || typeof d.alertedMatches !== "object") d.alertedMatches
  if (!d.alertedEvents || typeof d.alertedEvents !== "object") d.alertedEvents
  return d;
}

function saveData(d) { fs.writeFileSync(DATA_FILE, JSON.stringify(d, null, 2)); }

// — SMART NUMBER PARSER (German + English) —————
function smartParseNumber(str) {
  if (!str) return 0;
  const s = str.trim();

```

```

const hasComma = s.includes(",");
const hasDot   = s.includes(".");
if (hasComma && hasDot) {
  if (s.lastIndexOf(",") > s.lastIndexOf(".")) {
    return parseFloat(s.replace(/\. /g, "").replace(",", "."));
  } else {
    return parseFloat(s.replace(/, /g, ""));
  }
}
if (hasComma) {
  const parts = s.split(",");
  if (parts[parts.length - 1].length <= 2) return parseFloat(s.replace(",", ".")
  return parseFloat(s.replace(/, /g, ""));
}
if (hasDot) {
  const parts = s.split(".");
  if (parts[parts.length - 1].length <= 2) return parseFloat(s);
  return parseFloat(s.replace(/\. /g, ""));
}
return parseFloat(s);
}

```

// — PARSE FAHRLY/UBER REPORT —————

```

function parseReport(text) {
  const num = (label) => {
    const re = new RegExp(label + "\\s*:?\\s*(-?[\\d.,]+)\\s*€?", "i");
    const m = text.match(re);
    if (!m) return 0;
    return smartParseNumber(m[1]);
  };
  const dateRange = text.match(/(\\d+\\s+\\w+)\\s*[-]\\s*(\\d+\\s+\\w+)/);
  return {
    period: dateRange ? `${dateRange[1]} - ${dateRange[2]}` : "Unknown period",
    fahrten: parseInt(num("Fahrten")) || 0,
    netto:   num("Netto-Fahrpreis"),
    aktionen: num("Aktionen"),
    trinkgeld: num("Trinkgeld"),
    gesamt:   num("Gesamtumsätze"),
    bargeld: Math.abs(num("Eingenommenes Bargeld")),
    addedAt: new Date().toISOString(),
  };
}

```

```

const fmt = (n) => `${n.toFixed(2).replace(".", ",")} €`;

```

// — DISTANCE FROM HERNE —————

```

function distanceKm(lat1, lon1, lat2, lon2) {

```

```

const R = 6371;
const dLat = (lat2-lat1) * Math.PI/180;
const dLon = (lon2-lon1) * Math.PI/180;
const a = Math.sin(dLat/2)**2 + Math.cos(lat1*Math.PI/180)*Math.cos(lat2*Math.PI/180);
return Math.round(R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a)));
}

```

// — CITIES + STREETS (~90 streets total) —————

```

const CITIES = [
  { id:"bochum", name:"Bochum", searchArea:"Bochum, Germany",
    lat:51.4818, lon:7.2197,
    streets:[
      { name:"Castroper Str / Ruhrstadion", lat:51.4900, lon:7.2358 },
      { name:"Jahrhunderthalle", lat:51.4863, lon:7.2089 },
      { name:"Kortumstraße", lat:51.4815, lon:7.2188 },
      { name:"Brüderstraße / Bermudadreieck", lat:51.4810, lon:7.2196 },
      { name:"Viktoriastraße", lat:51.4789, lon:7.2192 },
      { name:"Stühmeyerstraße", lat:51.4838, lon:7.2253 },
      { name:"Hbf Bochum", lat:51.4787, lon:7.2234 },
      { name:"Herner Straße", lat:51.4900, lon:7.2173 },
      { name:"Wittener Straße", lat:51.4793, lon:7.2390 },
      { name:"Universitätsstraße (RUB)", lat:51.4470, lon:7.2657 },
      { name:"Konrad-Adenauer-Platz", lat:51.4793, lon:7.2173 },
      { name:"Massenbergstraße", lat:51.4801, lon:7.2208 },
    ] },
  { id:"dortmund", name:"Dortmund", searchArea:"Dortmund, Germany",
    lat:51.5136, lon:7.4653,
    streets:[
      { name:"Strobelallee / Signal Iduna", lat:51.4925, lon:7.4519 },
      { name:"Westfalenhalle", lat:51.4978, lon:7.4548 },
      { name:"Brückstraße", lat:51.5141, lon:7.4684 },
      { name:"Kleppingstraße", lat:51.5142, lon:7.4669 },
      { name:"Hbf Dortmund", lat:51.5179, lon:7.4593 },
      { name:"Kronprinzenstraße", lat:51.5237, lon:7.4621 },
      { name:"Borsigplatz", lat:51.5247, lon:7.4768 },
      { name:"Kaiserstraße", lat:51.5169, lon:7.4711 },
      { name:"Hansaplatz", lat:51.5158, lon:7.4666 },
      { name:"Möllerbrücke", lat:51.5071, lon:7.4595 },
      { name:"U-Reinoldikirche", lat:51.5147, lon:7.4665 },
      { name:"Königswall", lat:51.5170, lon:7.4625 },
    ] },
  { id:"essen", name:"Essen", searchArea:"Essen, Germany",
    lat:51.4556, lon:7.0116,
    streets:[
      { name:"Rüttenscheider Straße", lat:51.4366, lon:7.0026 },
      { name:"Hbf Essen", lat:51.4512, lon:7.0139 },
      { name:"Kettwiger Straße", lat:51.4555, lon:7.0103 },
    ] }
]

```

```

    { name:"Viehofer Straße",          lat:51.4582, lon:7.0156 },
    { name:"Grugahalle",              lat:51.4376, lon:7.0179 },
    { name:"Limbecker Platz",         lat:51.4582, lon:7.0085 },
    { name:"Steeler Straße",          lat:51.4504, lon:7.0345 },
    { name:"Hindenburgstraße",        lat:51.4545, lon:7.0157 },
    { name:"Kennedyplatz",            lat:51.4562, lon:7.0123 },
    { name:"Messe Essen",             lat:51.4360, lon:7.0177 },
  ] },
{ id:"duisburg", name:"Duisburg", searchArea:"Duisburg, Germany",
  lat:51.4344, lon:6.7623,
  streets:[
    { name:"Königstraße",          lat:51.4341, lon:6.7610 },
    { name:"Hbf Duisburg",         lat:51.4314, lon:6.7748 },
    { name:"Düsseldorfer Str",     lat:51.4255, lon:6.7681 },
    { name:"MSV Arena",            lat:51.4082, lon:6.7779 },
    { name:"Innenhafen",          lat:51.4406, lon:6.7716 },
    { name:"Kuhstraße",            lat:51.4327, lon:6.7641 },
    { name:"Marientor",            lat:51.4328, lon:6.7592 },
  ] },
{ id:"duesseldorf", name:"Düsseldorf", searchArea:"Düsseldorf, Germany",
  lat:51.2277, lon:6.7735,
  streets:[
    { name:"Bolkerstraße / Altstadt", lat:51.2271, lon:6.7740 },
    { name:"Königsallee",            lat:51.2238, lon:6.7790 },
    { name:"Hbf Düsseldorf",         lat:51.2200, lon:6.7940 },
    { name:"Flughafen DUS",          lat:51.2895, lon:6.7668 },
    { name:"Merkur Spiel-Arena",      lat:51.2614, lon:6.7333 },
    { name:"Charlottenstraße",        lat:51.2272, lon:6.7891 },
    { name:"Mitsubishi Electric Halle",lat:51.2174, lon:6.7440 },
    { name:"Schadowstraße",          lat:51.2272, lon:6.7820 },
    { name:"Heinrich-Heine-Allee",    lat:51.2275, lon:6.7728 },
    { name:"Medienhafen",            lat:51.2179, lon:6.7546 },
    { name:"Burgplatz",              lat:51.2278, lon:6.7705 },
    { name:"Friedrichstraße",        lat:51.2169, lon:6.7780 },
    { name:"Worringer Platz",         lat:51.2280, lon:6.7866 },
  ] },
{ id:"gelsenkirchen", name:"Gelsenkirchen", searchArea:"Gelsenkirchen, Germany",
  lat:51.5177, lon:7.0857,
  streets:[
    { name:"Veltins-Arena",          lat:51.5547, lon:7.0676 },
    { name:"Bahnhofstraße GE",       lat:51.5071, lon:7.1015 },
    { name:"Hbf Gelsenkirchen",      lat:51.5052, lon:7.1022 },
    { name:"Buer Mitte",             lat:51.5779, lon:7.1014 },
    { name:"Rheinelbestraße",        lat:51.5071, lon:7.0908 },
    { name:"Schalker Markt",         lat:51.5283, lon:7.0746 },
  ] },
{ id:"oberhausen", name:"Oberhausen", searchArea:"Oberhausen, Germany",

```

```

lat:51.4708, lon:6.8513,
streets:[
  { name:"CentrO", lat:51.4943, lon:6.8762 },
  { name:"König-Pilsener-Arena", lat:51.4926, lon:6.8772 },
  { name:"Hbf Oberhausen", lat:51.4736, lon:6.8519 },
  { name:"Marktstraße", lat:51.4719, lon:6.8540 },
  { name:"Friedensplatz", lat:51.4705, lon:6.8523 },
  { name:"Sterkrade Bf", lat:51.5159, lon:6.8503 },
] },
{ id:"muelheim", name:"Mülheim", searchArea:"Mülheim an der Ruhr, Germany",
lat:51.4275, lon:6.8826,
streets:[
  { name:"Hbf Mülheim", lat:51.4314, lon:6.8830 },
  { name:"Schloßstraße", lat:51.4305, lon:6.8855 },
  { name:"Forum City", lat:51.4326, lon:6.8828 },
] },
{ id:"hagen", name:"Hagen", searchArea:"Hagen, Germany",
lat:51.3671, lon:7.4633,
streets:[
  { name:"Hbf Hagen", lat:51.3667, lon:7.4624 },
  { name:"Elberfelder Straße", lat:51.3613, lon:7.4715 },
  { name:"Volme Galerie", lat:51.3622, lon:7.4649 },
] },
{ id:"wuppertal", name:"Wuppertal", searchArea:"Wuppertal, Germany",
lat:51.2562, lon:7.1508,
streets:[
  { name:"Hbf Wuppertal", lat:51.2549, lon:7.1495 },
  { name:"Luisenstraße", lat:51.2586, lon:7.1438 },
  { name:"Kipdorf", lat:51.2575, lon:7.1500 },
] },
{ id:"herne", name:"Herne", searchArea:"Herne, Germany",
lat:51.5393, lon:7.2261,
streets:[
  { name:"Hbf Herne", lat:51.5393, lon:7.2261 },
  { name:"Wanne-Eickel Hbf", lat:51.5316, lon:7.1635 },
  { name:"Cranger Kirmes", lat:51.5364, lon:7.1503 },
  { name:"Bahnhofstraße", lat:51.5378, lon:7.2258 },
] },
{ id:"recklinghausen", name:"Recklinghausen", searchArea:"Recklinghausen, Germa
lat:51.6135, lon:7.1972,
streets:[
  { name:"Hbf Recklinghausen", lat:51.6135, lon:7.1815 },
  { name:"Altstadt", lat:51.6147, lon:7.1972 },
  { name:"Ruhrfestspielhaus", lat:51.6232, lon:7.1922 },
] },
{ id:"witten", name:"Witten", searchArea:"Witten, Germany",
lat:51.4434, lon:7.3357,

```

```

    streets:[
      { name:"Hbf Witten", lat:51.4387, lon:7.3327 },
      { name:"Ruhrstraße", lat:51.4438, lon:7.3349 },
      { name:"Innenstadt", lat:51.4434, lon:7.3357 },
    ] },
{ id:"marl", name:"Marl", searchArea:"Marl, Germany",
  lat:51.6571, lon:7.0908,
  streets:[
    { name:"Marler Stern", lat:51.6571, lon:7.0908 },
    { name:"Hbf Marl-Sinsen",lat:51.6772, lon:7.1450 },
  ] },
{ id:"hattingen", name:"Hattingen", searchArea:"Hattingen, Germany",
  lat:51.3994, lon:7.1857,
  streets:[
    { name:"Altstadt", lat:51.3994, lon:7.1857 },
    { name:"Hbf Hattingen",lat:51.4042, lon:7.1696 },
  ] },
{ id:"castrop", name:"Castrop-Rauxel", searchArea:"Castrop-Rauxel, Germany",
  lat:51.5503, lon:7.3107,
  streets:[
    { name:"Stadtmitte", lat:51.5503, lon:7.3107 },
    { name:"Hbf Castrop Süd",lat:51.5379, lon:7.3094 },
  ] },
{ id:"unna", name:"Unna", searchArea:"Unna, Germany",
  lat:51.5365, lon:7.6890,
  streets:[
    { name:"Altstadt / Markt", lat:51.5365, lon:7.6890 },
    { name:"Hbf Unna", lat:51.5346, lon:7.6968 },
  ] },
];

// Compute distance from Herne for each city
for (const c of CITIES) {
  c.distFromHerne = distanceKm(HOME_BASE.lat, HOME_BASE.lon, c.lat, c.lon);
}

const AIRPORTS = [
  { code:"DUS", name:"Düsseldorf", affectedCities:["duesseldorf","duisburg","o
  { code:"DTM", name:"Dortmund", affectedCities:["dortmund","unna","castrop"
  { code:"CGN", name:"Köln/Bonn", affectedCities:["duesseldorf"] },
];

// — VENUE → CITY MAPPING for events —————
// External venues outside our 17 cities map to their NEAREST tracked city so
// their surge effect still scores.
const VENUE_TO_CITY = {
  "westfalenhalle": "dortmund",

```

```

"signal iduna": "dortmund",
"veltins-arena": "gelsenkirchen",
"veltins arena": "gelsenkirchen",
"merkur spiel-arena": "duesseldorf",
"merkur spiel arena": "duesseldorf",
"mitsubishi electric halle": "duesseldorf",
"mitsubishi": "duesseldorf",
"psd bank dome": "duesseldorf",
"jahrhunderthalle": "bochum",
"ruhrcongress": "bochum",
"könig-pilsener-arena": "oberhausen",
"könig pilsener arena": "oberhausen",
"rudolf weber-arena": "oberhausen",
"lanxess arena": "duesseldorf",          // Köln → nearest tracked = Düsseldorf
"msv arena": "duisburg",
"schauinsland-reisen-arena": "duisburg",
"grugahalle": "essen",
"messe essen": "essen",
"stadion essen": "essen",
"ruhrfestspielhaus": "recklinghausen",
"tonhalle": "duesseldorf",
"stadthalle wuppertal": "wuppertal",
"historische stadthalle": "wuppertal",
"bayarena": "duesseldorf",              // Leverkusen → nearest tracked
};

```

```

function venueToCity(venueName) {
  if (!venueName) return null;
  const lower = venueName.toLowerCase();
  for (const [key, cityId] of Object.entries(VENUE_TO_CITY)) {
    if (lower.includes(key)) return cityId;
  }
  return null;
}

```

```

// ——— STATE —————
const prevLevels = {};
const recentFlights = {};
let upcomingMatches = [];
let upcomingEvents = [];
let activeStrikes = [];

// Pre-compute distance from Herne for every city (used in alerts and reports)
for (const c of CITIES) {
  if (typeof c.lat === "number" && typeof c.lon === "number") {
    c.distFromHerne = distanceKm(HOME_BASE.lat, HOME_BASE.lon, c.lat, c.lon);
  } else {

```

```

        c.distFromHerne = 0;
    }
}

// ——— PEAK HOUR DETECTION —————
// أوقات الذروة المخصصة لسيدو:
// 04:00-07:00 (الصباح المبكر)
// 10:00-14:00 (الظهر)
// 16:00-18:00 (العصر/الخروج من الشغل)
// 20:30-23:30 (الليل/النوادي)
function isPeakHour() {
    const now = new Date();
    const h = now.getHours();
    const m = now.getMinutes();
    const hm = h + m/60;
    if (hm >= 4.0 && hm < 7.0) return true; // 04:00 - 07:00
    if (hm >= 10.0 && hm < 14.0) return true; // 10:00 - 14:00
    if (hm >= 16.0 && hm < 18.0) return true; // 16:00 - 18:00
    if (hm >= 20.5 && hm < 23.5) return true; // 20:30 - 23:30
    return false;
}

// ——— TOMTOM —————
async function fetchTraffic(lat, lon) {
    try {
        const url = `https://api.tomtom.com/traffic/services/4/flowSegmentData/absolu
        const r = await fetch(url);
        if (!r.ok) return null;
        const d = await r.json();
        const seg = d.flowSegmentData;
        if (!seg) return null;
        return { current: seg.currentSpeed, free: seg.freeFlowSpeed, ratio: seg.curre
    } catch { return null; }
}

async function fetchWeather(lat, lon) {
    try {
        const r = await fetch(`https://api.open-meteo.com/v1/forecast?latitude=${lat}
        const d = await r.json();
        return {
            rain: d.current?.precipitation > 0.2,
            mm: d.current?.precipitation || 0,
            temp: d.current?.temperature_2m,
            tomorrowRain: d.daily?.precipitation_sum?.[1] || 0,
        };
    } catch { return { rain: false, mm: 0, temp: null, tomorrowRain: 0 }; }
}

```



```
// — AERODATABOX (Flights) —————
async function fetchArrivals(airportCode) {
  if (!RAPIDAPI_KEY) return [];
  const now = new Date();
  const past = new Date(now.getTime() - 60*60*1000);
  const fmt = (d) => d.toISOString().slice(0,16);
  const url = `https://aerodatabox.p.rapidapi.com/flights/airports/iata/${airportCode}`;
  const r = await fetch(url, {
    headers: { "X-RapidAPI-Key": RAPIDAPI_KEY, "X-RapidAPI-Host": "aerodatabox.p.rapidapi.com" };
  });
  if (!r.ok) throw new Error(`AeroDataBox ${r.status}`);
  const d = await r.json();
  return (d.arrivals || []).map(f => ({
    flight: f.number || "?",
    origin: f.movement?.airport?.name || "?",
    time: f.movement?.actualTime?.local || f.movement?.scheduledTime?.local,
  }));
}
```

```
// — FOOTBALL-DATA (Matches) —————
// Teams playing at home (we care about home matches only – that's where the surge is)
// For teams outside our tracked CITIES (Leverkusen, Köln, Mönchengladbach), we
// map them to the NEAREST tracked city so their post-match surge still scores.
const RUHR_TEAMS = {
  // teamId -> {name, venue, cityId, externalCity?}
  4: { name:"Borussia Dortmund", venue:"Signal Iduna Park", cityId:"Dortmund", externalCity:"Dortmund" },
  6: { name:"FC Schalke 04", venue:"Veltins-Arena", cityId:"Gelsenkirchen", externalCity:"Gelsenkirchen" },
  3: { name:"Bayer Leverkusen", venue:"BayArena", cityId:"Leverkusen", externalCity:"Leverkusen" },
  18: { name:"Borussia Mönchengladbach", venue:"Borussia-Park", cityId:"Mönchengladbach", externalCity:"Mönchengladbach" },
  16: { name:"Fortuna Düsseldorf", venue:"Merkur Spiel-Arena", cityId:"Düsseldorf", externalCity:"Düsseldorf" },
  36: { name:"VfL Bochum", venue:"Vonovia Ruhrstadion", cityId:"Bochum", externalCity:"Bochum" },
  28: { name:"FC Köln", venue:"RheinEnergieStadion", cityId:"Köln", externalCity:"Köln" },
  11: { name:"MSV Duisburg", venue:"MSV-Arena", cityId:"Duisburg", externalCity:"Duisburg" },
};
```

```
async function fetchUpcomingMatches() {
  if (!FOOTBALL_KEY) return [];
  const now = new Date();
  const future = new Date(now.getTime() + 7*24*60*60*1000);
  const dateFrom = now.toISOString().slice(0,10);
  const dateTo = future.toISOString().slice(0,10);

  // Bundesliga, 2.Bundesliga, Champions League, DFB-Pokal, Europa League
  const competitions = ["BL1","BL2","CL","DFB","EL"];
  const matches = [];
  for (const comp of competitions) {
```

```

try {
  const url = `https://api.football-data.org/v4/competitions/${comp}/matches?`;
  const r = await fetch(url, { headers: { "X-Auth-Token": FOOTBALL_KEY } });
  if (!r.ok) continue;
  const d = await r.json();
  for (const m of (d.matches || [])) {
    const homeId = m.homeTeam?.id;
    const team = RUHR_TEAMS[homeId];
    if (!team) continue;
    matches.push({
      id: m.id,
      competition: m.competition?.name || comp,
      home: m.homeTeam?.name,
      away: m.awayTeam?.name,
      time: m.utcDate,
      venue: team.venue,
      cityId: team.cityId,           // surge city (mapped to nearest tracked
      externalCity: team.externalCity || null, // real city if outside track
      status: m.status,
    });
  }
} catch(e) { console.error(`Football ${comp}: ${e.message}`); }
}

return matches.sort((a,b) => new Date(a.time) - new Date(b.time));
}

```

```

// ——— TICKETMASTER (Concerts/Events) —————
async function fetchUpcomingEvents() {
  if (!TICKETMASTER_KEY) return [];
  const now = new Date();
  const future = new Date(now.getTime() + 7*24*60*60*1000);
  const startISO = now.toISOString().slice(0,19) + "Z";
  const endISO = future.toISOString().slice(0,19) + "Z";

  // Search for events in NRW region
  const url = `https://app.ticketmaster.com/discovery/v2/events.json?apikey=${TIC
  try {
    const r = await fetch(url);
    if (!r.ok) throw new Error(`TM ${r.status}`);
    const d = await r.json();
    const events = (d._embedded?.events || []).map(ev => {
      const venueName = ev._embedded?.venues?.[0]?.name || "";
      const venueCity = ev._embedded?.venues?.[0]?.city?.name || "";
      const cityId = venueToCity(venueName) || venueToCity(venueCity);
      return {
        id: ev.id,
        name: ev.name,

```

```

        venue: venueName,
        city: venueCity,
        cityId,
        time: ev.dates?.start?.dateTime || ev.dates?.start?.localDate,
    };
    })).filter(e => e.cityId); // only events we can map to a city
    return events;
} catch(e) {
    console.error(`Ticketmaster: ${e.message}`);
    return [];
}
}

// — VRR / DB STRIKES & DISRUPTIONS —————
// Uses transport.rest (free, no key) for DB delays
async function fetchTransportDisruptions() {
    // We check known major hubs in Ruhrgebiet and report disruptions
    const stations = [
        { id:"8000080", name:"Dortmund Hbf", cityId:"dortmund" },
        { id:"8000098", name:"Essen Hbf", cityId:"essen" },
        { id:"8000044", name:"Bochum Hbf", cityId:"bochum" },
        { id:"8000086", name:"Düsseldorf Hbf", cityId:"duesseldorf" },
        { id:"8000084", name:"Duisburg Hbf", cityId:"duisburg" },
    ];
    const disruptions = [];
    for (const st of stations) {
        try {
            const r = await fetch(`https://v6.db.transport.rest/stops/${st.id}/departur
            if (!r.ok) continue;
            const d = await r.json();
            const dep = d.departures || [];
            const cancelled = dep.filter(x => x.cancelled).length;
            const delayed = dep.filter(x => x.delay && x.delay > 600).length; // 10+
            if (cancelled >= 3 || delayed >= 5) {
                disruptions.push({
                    station: st.name,
                    cityId: st.cityId,
                    cancelled, delayed,
                });
            }
        } catch(e) { /* ignore */ }
    }
    return disruptions;
}

// Strike detection – we check VRR site for keyword "Streik" via simple fetch
async function fetchStrikeNews() {

```

```

try {
  const r = await fetch("https://www.vrr.de/de/service/stoerungen-und-aktuelles");
  if (!r.ok) return [];
  const text = await r.text();
  const strikes = [];
  if (/streik/i.test(text)) {
    strikes.push({
      title: "إضراب محتمل في VRR",
      source: "vrr.de",
      time: new Date().toISOString(),
    });
  }
  return strikes;
} catch { return []; }
}

```

```

// — CITY SCAN —
async function scanCityStreets(city) {
  const results = [];
  let totalRatio = 0, count = 0;
  for (const st of city.streets) {
    const tf = await fetchTraffic(st.lat, st.lon);
    if (tf) {
      totalRatio += tf.ratio;
      count++;
      let status = "FREE";
      if (tf.ratio < 0.4) status = "JAM";
      else if (tf.ratio < 0.6) status = "SLOW";
      else if (tf.ratio < 0.85) status = "MODERATE";
      results.push({ name: st.name, speed: Math.round(tf.current), pct: Math.round(
    }
  }

  const weather = await fetchWeather(city.streets[0].lat, city.streets[0].lon);
  const avgRatio = count > 0 ? totalRatio / count : 1;
  let score = 3;
  if (avgRatio < 0.4) score += 4;
  else if (avgRatio < 0.6) score += 3;
  else if (avgRatio < 0.85) score += 1;
  if (weather.rain) score += 1;
  const hour = new Date().getHours(), day = new Date().getDay();
  if ((day === 5 || day === 6) && (hour >= 22 || hour < 4)) score += 2;
  else if ((day === 5 || day === 6) && hour >= 18) score += 1;
  else if (hour >= 17 && hour <= 19) score += 1;
  for (const ap of AIRPORTS) {
    if (ap.affectedCities.includes(city.id)) {
      const flights = recentFlights[ap.code] || [];
      if (flights.length >= 5) score += 2;
    }
  }
}

```

```

        else if (flights.length >= 2) score += 1;
    }
}
// Boost for events in this city today
const today = new Date().toDateString();
const eventsToday = upcomingEvents.filter(e => e.cityId === city.id && new Date
const matchesToday = upcomingMatches.filter(m => m.cityId === city.id && new Da
if (eventsToday.length || matchesToday.length) score += 2;
// Boost for strikes in this city
const strikesHere = activeStrikes.filter(s => !s.cityId || s.cityId === city.id
if (strikesHere.length) score += 1;
score = Math.min(10, Math.max(1, score));
const level = score >= 8 ? "CRITICAL" : score >= 6 ? "HIGH" : score >= 4 ? "MED
results.sort((a,b) => a.pct - b.pct);
return { score, level, streets: results, weather, avgRatio: Math.round(avgRatio
}

// — TRAFFIC SCAN — only during peak hours —————
async function trafficScan() {
    if (!isPeakHour()) {
        console.log(`[${new Date().toLocaleTimeString("de-DE")}] Skipped (off-peak)`)
        return;
    }
    console.log(`[${new Date().toLocaleTimeString("de-DE")}] Traffic scan running (
    for (const city of CITIES) {
        try {
            const data = await scanCityStreets(city);
            const isHigh = data.level === "HIGH" || data.level === "CRITICAL";
            const wasLow = !prevLevels[city.id] || prevLevels[city.id] === "LOW" || pre
            if (isHigh && wasLow) {
                const icon = data.level === "CRITICAL" ? "🔴" : "🟡";
                const jams = data.streets.filter(r => r.status === "JAM" || r.status ===
                    .map(r => `📍 ${r.name} — ${r.speed}km/h (${r.pct}%)`).joi
                const rain = data.weather.rain ? `🌧️ مطر: ${data.weather.mm}mm` : "";
                await bot.sendMessage(CHAT_ID, `${icon} *${city.name} — ${data.level} (${
            }
            prevLevels[city.id] = data.level;
        } catch(e) { console.error(`${city.name}: ${e.message}`); }
    }
}

// — FLIGHT SCAN —————
async function flightScan() {
    for (const ap of AIRPORTS) {
        try {
            const flights = await fetchArrivals(ap.code);
            recentFlights[ap.code] = flights;

```

```

    if (flights.length >= 3) {
      const data = loadData();
      const key = `${ap.code}_${flights.length}_${new Date().getHours()}`;
      if (data.alertedFlights[key]) continue;
      data.alertedFlights[key] = Date.now();
      // Cleanup old keys (>4h)
      for (const k of Object.keys(data.alertedFlights)) {
        if (Date.now() - data.alertedFlights[k] > 4*60*60*1000) delete data.alertedFlights[k];
      }
      saveData(data);
      const cityNames = ap.affectedCities.map(id => CITIES.find(c => c.id === id));
      const list = flights.slice(0,5).map(f => `✈️ ${f.flight} ${f.origin}`).join(' ');
      await bot.sendMessage(CHAT_ID, `👋 *${ap.name} (${ap.code})* - ${flights.length} flights`);
    } catch(e) { console.error(`${ap.code}: ${e.message}`); }
  }
}

// — MATCH ALERTS —
async function matchAlertScan() {
  upcomingMatches = await fetchUpcomingMatches();
  const data = loadData();
  const now = Date.now();
  for (const m of upcomingMatches) {
    const matchTime = new Date(m.time).getTime();
    const diff = matchTime - now;
    const city = CITIES.find(c => c.id === m.cityId);
    const dist = city ? city.distFromHerne : "?";
    // For external teams (Köln/Leverkusen/Mönchengladbach), show real city name
    const realCityName = m.externalCity || city?.name || m.cityId;
    const surgeNote = m.externalCity ? `⚡️ (الضغط متوقع في) ${city?.name} || "Düsseldorf"` : "";

    // Alert 2 hours before kickoff (window: 1h55m to 2h05m to be safe with 5-min window)
    if (diff > 1*60*60*1000 + 55*60*1000 && diff < 2*60*60*1000 + 5*60*1000) {
      const key = `pre_${m.id}`;
      if (!data.alertedMatches[key]) {
        data.alertedMatches[key] = now;
        const localTime = new Date(m.time).toLocaleString("de-DE", { hour:"2-digit", minute:"2-digit" });
        await bot.sendMessage(CHAT_ID, `⚽️ *مباراة بعد ساعتين*\n\n${m.home} ضد ${m.away}\n🕒 ${localTime}\n🇩🇪`);
        { parse_mode:"Markdown" });
      }
    }

    // Alert when match likely just ended (~105-120 min after start, wide window)
    const endDiff = now - matchTime;
    if (endDiff > 105*60*1000 && endDiff < 120*60*1000) {
      const key = `post_${m.id}`;

```

```

        if (!data.alertedMatches[key]) {
            data.alertedMatches[key] = now;
            await bot.sendMessage(CHAT_ID,
                `🏠 *الآن* انتهت - المباراة!*\n\n${m.home} ضد ${m.away}\n🏟️ ${m.venue}`
                { parse_mode:"Markdown" });
        }
    }
}

// Cleanup old alerts (>24h)
for (const k of Object.keys(data.alertedMatches)) {
    if (now - data.alertedMatches[k] > 24*60*60*1000) delete data.alertedMatches[k]
}
saveData(data);
}

// ——— EVENT ALERTS ———
async function eventAlertScan() {
    upcomingEvents = await fetchUpcomingEvents();
    const data = loadData();
    const now = Date.now();
    for (const ev of upcomingEvents) {
        const eventTime = new Date(ev.time).getTime();
        const diff = eventTime - now;
        if (diff > 0 && diff < 60*60*1000 + 5*60*1000) {
            const key = `pre_${ev.id}`;
            if (data.alertedEvents[key]) continue;
            data.alertedEvents[key] = now;
            const city = CITIES.find(c => c.id === ev.cityId);
            const dist = city ? city.distFromHerne : "?";
            const localTime = new Date(ev.time).toLocaleString("de-DE", { hour:"2-digit" });
            await bot.sendMessage(CHAT_ID,
                `🎵 *حدث بعد ساعة*\n\n${ev.name}\n🕒 ${localTime}\n🏠 ${ev.venue}\n📍 ${city ? city.name : "?"}`
                { parse_mode:"Markdown" });
        }
    }
}

for (const k of Object.keys(data.alertedEvents)) {
    if (now - data.alertedEvents[k] > 24*60*60*1000) delete data.alertedEvents[k]
}
saveData(data);
}

// ——— STRIKE / TRANSPORT ALERTS ———
async function disruptionScan() {
    const data = loadData();
    const now = Date.now();

    // Strike news (broad)

```

```

const strikes = await fetchStrikeNews();
activeStrikes = strikes;
for (const s of strikes) {
  const key = `strike_${s.title}`;
  if (data.alertedEvents[key]) continue;
  data.alertedEvents[key] = now;
  await bot.sendMessage(CHAT_ID, `🚩 *تنبيه إضراب*\n\n${s.title}\n📰 المصدر: ${s.source}`);
}

// Train disruptions
const dis = await fetchTransportDisruptions();
for (const d of dis) {
  const key = `dis_${d.station}_${new Date().getHours()}`;
  if (data.alertedEvents[key]) continue;
  data.alertedEvents[key] = now;
  const city = CITIES.find(c => c.id === d.cityId);
  const dist = city ? city.distFromHerne : "?";
  await bot.sendMessage(CHAT_ID, `🚆 *تعطل في القطارات*\n\n📍 ${d.station} (${dist} كم)\n❌ ملغية: ${d.cancelled}`);
  { parse_mode:"Markdown" });
}

saveData(data);
}

// — DAILY REPORT (21:00) —————
async function buildDailyReport() {
  const now = new Date();
  const tomorrow = new Date(now.getTime() + 24*60*60*1000);
  const tomorrowDate = tomorrow.toDateString();
  const dayName = tomorrow.toLocaleDateString("de-DE", { weekday:"long" });
  const dateStr = tomorrow.toLocaleDateString("de-DE", { day:"numeric", month:"long" });
  const isWeekend = tomorrow.getDay() === 5 || tomorrow.getDay() === 6;

  // Refresh data
  upcomingMatches = await fetchUpcomingMatches();
  upcomingEvents = await fetchUpcomingEvents();
  const strikes = await fetchStrikeNews();
  const weather = await fetchWeather(HOME_BASE.lat, HOME_BASE.lon);

  // Filter for tomorrow
  const matchesT = upcomingMatches.filter(m => new Date(m.time).toDateString() === tomorrowDate);
  const eventsT = upcomingEvents.filter(e => new Date(e.time).toDateString() === tomorrowDate);

  // Score each city for tomorrow
  const cityScores = CITIES.map(c => {
    let score = 0;

```



```

const reasons = [];
// Events
const cityMatches = matchesT.filter(m => m.cityId === c.id);
const cityEvents = eventsT.filter(e => e.cityId === c.id);
if (cityMatches.length) { score += 4; reasons.push(`⚽ ${cityMatches.length}`);
if (cityEvents.length) { score += 3; reasons.push(`🎵 ${cityEvents.length}`);
// Airports
for (const ap of AIRPORTS) {
  if (ap.affectedCities.includes(c.id)) score += 1;
}
// Distance from Herne (closer = better)
if (c.distFromHerne < 15) score += 2;
else if (c.distFromHerne < 30) score += 1;
// Weekend boost
if (isWeekend && (c.id === "duesseldorf" || c.id === "dortmund" || c.id === "
// Strikes
if (strikes.length) score += 1;
return { city: c, score, reasons, matches: cityMatches, events: cityEvents };
}).sort((a,b) => b.score - a.score);

// Build report
let text = `🇩🇪 * تقرير الغد - ${dayName} ${dateStr}*\\n 📍 من Herne\\n\\n`;

// Weather
if (weather.tomorrowRain > 1) {
  text += `☁️ * مطر: الطقس: (${weather.tomorrowRain.toFixed(1)}mm)\\n لب أعلى
} else {
  text += `☀️ * جيد: الطقس:\\n\\n`;
}

// Matches
if (matchesT.length) {
  text += `⚽ المباريات (${matchesT.length})*\\n`;
  for (const m of matchesT.slice(0,5)) {
    const city = CITIES.find(c => c.id === m.cityId);
    const dist = city ? `${city.distFromHerne} كم` : "?";
    const time = new Date(m.time).toLocaleString("de-DE", { hour:"2-digit", min
    const venueLabel = m.externalCity ? `${m.venue} - ${m.externalCity}` : m.ve
    text += `• ${time} - ${m.home} ضد ${m.away}\\n 🏟️ ${venueLabel} (${dist})\\
  }
  text += `\\n`;
}

// Events
if (eventsT.length) {
  text += `🎵 الكونسيرترات والأحداث (${eventsT.length})*\\n`;
  for (const ev of eventsT.slice(0,5)) {

```

```

    const city = CITIES.find(c => c.id === ev.cityId);
    const dist = city ? `${city.distFromHerne} كم` : "?";
    const time = new Date(ev.time).toLocaleString("de-DE", { hour:"2-digit", mi
    text += `· ${time} - ${ev.name}\n 🏟️ ${ev.venue} (${dist})\n`;
  }
  text += `\n`;
}

// Strikes
if (strikes.length) {
  text += ` 🚩 *إضرابات محتملة:*\n`;
  for (const s of strikes) text += `· ${s.title}\n`;
  text += `\n`;
}

// Top cities recommendation
text += ` _____\n`;
text += ` 🎯 *أفضل المدن للغد:*\n`;
text += ` _____\n\n`;
const top = cityScores.filter(s => s.score > 0).slice(0,5);
if (!top.length) {
  text += ` _:_لا توجد أحداث كبيرة. الفوكس على الذروة العادية\n`;
  text += ` 🕒 16:00-18:00 - Bochum/Dortmund\n`;
  text += ` 🌙 20:30-23:30 - Düsseldorf/Bochum\n`;
} else {
  const medals = ["🥇", "🥈", "🥉", "4", "5"];
  top.forEach((s, i) => {
    text += `${medals[i]} *${s.city.name}* (${s.city.distFromHerne} كم)\n`;
    if (s.reasons.length) text += `   ${s.reasons.join(" · ")}\n`;
    if (s.matches.length) {
      const m = s.matches[0];
      const time = new Date(m.time).toLocaleString("de-DE", { hour:"2-digit", m
      text += `   ⚽ ${time} ${m.home}\n`;
    }
    if (s.events.length) {
      const ev = s.events[0];
      const time = new Date(ev.time).toLocaleString("de-DE", { hour:"2-digit",
      text += `   🎵 ${time} ${ev.name.slice(0,40)}\n`;
    }
    text += `\n`;
  });
}

// Recommended schedule
text += ` _____\n`;
text += ` 🕒 *خطة اليوم المقترحة:*\n`;
text += ` _____\n`;

```

```

text += `🌆 04:00-07:00 - Herne/Bochum\n`;
text += `☀️ 10:00-14:00 - ${top[0]?.city.name || "Düsseldorf"}\n`;
text += `🏢 16:00-18:00 - ${top[1]?.city.name || "Dortmund"}\n`;
text += `🌙 20:30-23:30 - ${top[0]?.city.name || "Düsseldorf"}\n`;

await bot.sendMessage(CHAT_ID, text, { parse_mode:"Markdown" });
}

// Schedule daily report at 21:00 every day (robust against restarts)
function scheduleDailyReport() {
  const scheduleNext = () => {
    const now = new Date();
    const next = new Date(now);
    next.setHours(21, 0, 0, 0);
    if (next.getTime() <= now.getTime()) {
      // 21:00 today already passed → schedule for tomorrow
      next.setDate(next.getDate() + 1);
    }
    const delay = next.getTime() - now.getTime();
    console.log(`Daily report scheduled for ${next.toLocaleString("de-DE")} (in $
    setTimeout(async () => {
      try { await buildDailyReport(); }
      catch(e) { console.error(`Daily report: ${e.message}`); }
      scheduleNext(); // re-schedule for next day
    }, delay);
  };
  scheduleNext();
}

// — APIFY —————
async function runApifyScrape(searchArea, categories, maxPlaces = 5) {
  const url = `https://api.apify.com/v2/acts/compass~crawler-google-places/run-sy
  const r = await fetch(url, {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      searchStringsArray: categories, locationQuery: searchArea,
      maxCrawledPlacesPerSearch: maxPlaces, language: "en",
      scrapePlaceDetailPage: true, scrapePopularTimesInsights: true,
    }),
  });
  if (!r.ok) throw new Error(`Apify ${r.status}`);
  return await r.json();
}

async function scrapeOneCity(city, chatId) {
  const items = await runApifyScrape(city.searchArea, ["bar","club","restaurant"]
  if (!items?.length) { await bot.sendMessage(chatId, `*${city.name}* : أماكن

```

```

const sorted = items.map(p => ({
  name: p.title, category: p.categoryName || "",
  livePct: p.popularTimesLivePercent || null,
  rating: p.totalScore,
})).sort((a,b) => (b.livePct||0) - (a.livePct||0));
let text = `👥 *${city.name}:*\n\n`;
for (const p of sorted.slice(0,6)) {
  if (p.livePct) {
    const bar = "█".repeat(Math.round(p.livePct/10)) + "░".repeat(10 - Math.round(p.livePct/10));
    text += `📍 *${p.name}*\n    ${p.category}\n    ${bar} ${p.livePct}%\n\n`;
  } else {
    text += `📍 *${p.name}*\n    ${p.category}${p.rating ? ` ⭐ ${p.rating}` : ''}`;
  }
}
await bot.sendMessage(chatId, text, { parse_mode:"Markdown" });
}

// — ARABIC FINANCE COMMANDS —————
async function handleDriverReport(msg, driver) {
  const text = msg.text;
  const data = loadData();
  const parsed = parseReport(text);
  if (parsed.gesamt === 0) {
    await bot.sendMessage(msg.chat.id,
      `❌ التقرير بهذا الشكل ما قدرت أقرأ التقرير\n\n*${driver}*\n6 Apr -
      { parse_mode:"Markdown" });
    return;
  }
  data[driver].push(parsed);
  saveData(data);
  await bot.sendMessage(msg.chat.id,
    `✅ تم تسجيل تقرير *${driver}*\n\n📅 ${parsed.period}\n🚗 رحلات: ${parsed.fah:
    { parse_mode:"Markdown" });
}

async function handleExpense(msg, type, label) {
  const text = msg.text;
  const m = text.match(/([\d.]+)/);
  if (!m) {
    await bot.sendMessage(msg.chat.id, `❌ مثال: \`${type} 60\``, { parse_mode:"M
    return;
  }
  const amount = smartParseNumber(m[1]);
  const data = loadData();
  data.expenses.push({ type, amount, date: new Date().toISOString() });
  saveData(data);
  await bot.sendMessage(msg.chat.id, `✅ تم تسجيل ${label}: ${fmt(amount)}\`);

```

```
}
```

```
async function handleReport(msg) {
  const data = loadData();
  const now = new Date();
  const monthStart = new Date(now.getFullYear(), now.getMonth(), 1).toISOString()
  const salam = data.salam.filter(r => r.addedAt >= monthStart);
  const kawa = data.kawa.filter(r => r.addedAt >= monthStart);
  const expenses = data.expenses.filter(e => e.date >= monthStart);
  const sumDriver = (arr) => arr.reduce((s, r) => ({
    netto: s.netto + r.netto, aktionen: s.aktionen + r.aktionen,
    trinkgeld: s.trinkgeld + r.trinkgeld, gesamt: s.gesamt + r.gesamt,
    fahrten: s.fahrten + r.fahrten,
  })), { netto:0, aktionen:0, trinkgeld:0, gesamt:0, fahrten:0 });
  const S = sumDriver(salam);
  const K = sumDriver(kawa);
  const salamTax = (S.netto + S.aktionen) * TAX_RATE;
  const salamNet = S.gesamt - salamTax;
  const kawaShare = K.gesamt * KAWA_SHARE;
  const ownerFromKawa = K.gesamt - kawaShare;
  const kawaTax = (K.netto + K.aktionen) * TAX_RATE;
  const ownerFromKawaNet = ownerFromKawa - kawaTax;
  const expByType = expenses.reduce((acc, e) => { acc[e.type] = (acc[e.type] || 0)
  const fuel = expByType.fuel || 0;
  const repair = expByType.repair || 0;
  const wash = expByType.wash || 0;
  const totalExp = FIXED_COMPANY + FIXED_INSURANCE + fuel + repair + wash;
  const grossOwner = salamNet + ownerFromKawaNet;
  const netOwner = grossOwner - totalExp;
  const monthName = now.toLocaleDateString("de-DE", { month:"long", year:"numeric
  let text = `🇩🇪 *تقرير* ${monthName}*\\n\\n`;
  text += `_____\\n`;
  text += `👤 *سلام* (${S.fahrten} رحلة)\\n Netto: ${fmt(S.netto)}\\n Aktionen: ${
  text += `👤 *كاوا* (${K.fahrten} رحلة)\\n الإجمالي: ${fmt(K.gesamt)}\\n 40 كاوا
  text += `_____\\n👤 *المصاريف*\\n شركة: ${fmt(FIXED_COMPANY)}\\n سين
  text += `_____\\n💰 *النتيجة النهائية*\\n دخلك: ${fmt(grossOwner)}\\n
  await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
}
```

```
async function handleReset(msg) {
  saveData({ salam:[], kawa:[], expenses:[], strikes:[], alertedFlights:{}, alert
  await bot.sendMessage(msg.chat.id, "✅ تم مسح كل البيانات");
}
```

```
async function handleCash(msg) {
  const data = loadData();
  const now = new Date();
```

```

const monthStart = new Date(now.getFullYear(), now.getMonth(), 1).toISOString()
const salam = data.salam.filter(r => r.addedAt >= monthStart);
const kawa = data.kawa.filter(r => r.addedAt >= monthStart);
const expenses = data.expenses.filter(e => e.date >= monthStart);
const sum = (arr, key) => arr.reduce((s,r) => s + (r[key]||0), 0);
const S_gesamt = sum(salam, "gesamt");
const S_bargeld = sum(salam, "bargeld");
const S_tax = (sum(salam,"netto") + sum(salam,"aktionen")) * TAX_RATE;
const S_net = S_gesamt - S_tax;
const K_gesamt = sum(kawa, "gesamt");
const K_bargeld = sum(kawa, "bargeld");
const K_tax = (sum(kawa,"netto") + sum(kawa,"aktionen")) * TAX_RATE;
const ownerFromKawa = K_gesamt * (1 - KAWA_SHARE);
const ownerFromKawaNet = ownerFromKawa - K_tax;
const kawaShare = K_gesamt * KAWA_SHARE;
const expSum = expenses.reduce((s,e) => s + e.amount, 0);
const cashOnHand = (S_bargeld + K_bargeld) - kawaShare - expSum;
const grossOwner = S_net + ownerFromKawaNet;
const runningNet = grossOwner - expSum;
const monthName = now.toLocaleDateString("de-DE", { month:"long", year:"numeric" });
let text = `💰 ملخص الكاش - ${monthName}*\\n\\n`;
text += `\\n🇩🇪 *حتى الآن: *\\n دخل سلام: ${fmt(S_gesamt)}\\n حل كاو ا
text += `💰 *الكاش المقبوض: *\\n من سلام: ${fmt(S_bargeld)}\\n من كاو ا: ${fmt(K_b
text += `💰 *مصاريف هذا الشهر: *\\n بنزين/تصليح/غسيل: ${fmt(expSum)}\\n وا (%40
text += `\\n💰 *كاش لازم يكون معك الآن: *\\n *${fmt(cashOnHand)}*\\n
text += `📈 *صافي ربحك حتى الآن: *\\n *${fmt(runningNet)}*\\n _ (تأمين 500 €
await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
}

```

// — MESSAGE ROUTING —

```

bot.on("message", async (msg) => {
  if (!msg.text) return;
  const text = msg.text.trim();
  const lower = text.toLowerCase();

  if (text.startsWith("سلام")) return handleDriverReport(msg, "salam");
  if (text.startsWith("كاوا")) return handleDriverReport(msg, "kawa");
  if (text.startsWith("بنزين")) return handleExpense(msg, "fuel", "بنزين");
  if (text.startsWith("تصليح")) return handleExpense(msg, "repair", "تصليح");
  if (text.startsWith("غسيل")) return handleExpense(msg, "wash", "غسيل");
  if (text === "تقرير") return handleReport(msg);
  if (text === "كاش") return handleCash(msg);
  if (text === "مسح البيانات") return handleReset(msg);

  if (lower.startsWith("/salam")) { msg.text = text.replace(/^\\s*salam\\s*/i,"سلام\\n");
  if (lower.startsWith("/kawa")) { msg.text = text.replace(/^\\s*kawa\\s*/i,"كاوا\\n");
  if (lower.startsWith("/fuel")) { msg.text = text.replace(/^\\s*fuel/i,"بنزين");

```

```

if (lower.startsWith("/repair")) { msg.text = text.replace(/^\/repair/i, "تصليح"}
if (lower.startsWith("/wash")) { msg.text = text.replace(/^\/wash/i, "غسيل");
if (lower === "/report") return handleReport(msg);
if (lower === "/cash") return handleCash(msg);
if (lower === "/reset") return handleReset(msg);
if (lower === "/daily") return buildDailyReport();

if (text === "مساعدة" || text === "/help") {
  await bot.sendMessage(msg.chat.id,
    `📁 *كاوا - تسجيل دخل كاوا\سلام* - تسجيل دخلك*\n\nالأوامر العربية (مالية)
    📁 *English aliases*\n/salam /kawa /fuel /repair /wash /report /cash /rese
    📁 *Traffic & info*\n/start /status /scan /flights /matches /events /strik
    { parse_mode:"Markdown" });
}
});

// ——— TRAFFIC/INFO COMMANDS ———
bot.onText(/\/start/, async (msg) => {
  await bot.sendMessage(msg.chat.id,
    "👋 *NRW Surge Bot v7*\n\n🚗 *تلقائي*: *🚦 (ساعات الذروة فقط)
    { parse_mode:"Markdown" });
});

bot.onText(/\/cities/, async (msg) => {
  await bot.sendMessage(msg.chat.id, `*Cities:*\n${CITIES.map(c => `• \`${c.id}\`
});

bot.onText(/\/status/, async (msg) => {
  let text = "🇩🇪 *الحالة الحالية:*\n\n";
  const sorted = CITIES.map(c => ({ name:c.name, level:prevLevels[c.id] || "UNKNO
    .sort((a,b) => ({CRITICAL:4,HIGH:3,MEDIUM:2,LOW:1,UNKNOWN:
  for (const c of sorted) {
    const i = c.level === "CRITICAL" ? "🔴" : c.level === "HIGH" ? "🟡" : c.level
    text += `${i} ${c.name} - ${c.level}\n`;
  }
  await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
});

bot.onText(/\/scan(?:\s+(\w+))?/, async (msg, match) => {
  const arg = match[1]?.toLowerCase();
  if (!arg) { await bot.sendMessage(msg.chat.id, "Usage: `\/scan bochum` or `\/scan
  if (arg === "all") { await bot.sendMessage(msg.chat.id, "🔄 Scanning..."); await
  const city = CITIES.find(c => c.id === arg);
  if (!city) { await bot.sendMessage(msg.chat.id, `❌ Unknown: ${arg}`); return;
  await bot.sendMessage(msg.chat.id, `🔄 Scanning *${city.name}*...`, { parse_mode
  try {
    const d = await scanCityStreets(city);

```

```

const icon = d.level === "CRITICAL" ? "🔴" : d.level === "HIGH" ? "🟡" : d.le
let text = `${icon} *${city.name} - ${d.level} (${d.score}/10)*\n🚦 Flow: ${d
if (d.weather.rain) text += `\n☁️ مطر: ${d.weather.mm}mm`;
text += `\n\n*الشوارع:*\n`;
for (const s of d.streets) {
  const si = s.status === "JAM" ? "🔴" : s.status === "SLOW" ? "🟡" : s.statu
  text += `${si} ${s.name} - ${s.speed}km/h (${s.pct}%) \n`;
}
await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
} catch(e) { await bot.sendMessage(msg.chat.id, `❌ ${e.message}`); }
});

```

```

bot.onText(/\\/flights/, async (msg) => {
  await bot.sendMessage(msg.chat.id, "🕒 Checking arrivals...");
  await flightScan();
  let text = "✈️ *Recent arrivals:*\n\n";
  for (const ap of AIRPORTS) {
    const f = recentFlights[ap.code] || [];
    text += `*${ap.name} (${ap.code})*: ${f.length} flights\n`;
    for (const fl of f.slice(0,5)) text += ` ✈️ ${fl.flight} ${fl.origin}\n`;
    text += "\n";
  }
  await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
});

```

```

bot.onText(/\\/matches/, async (msg) => {
  await bot.sendMessage(msg.chat.id, "🕒 Loading matches...");
  upcomingMatches = await fetchUpcomingMatches();
  if (!upcomingMatches.length) {
    await bot.sendMessage(msg.chat.id, "لا مباريات قادمة في 7 أيام", { parse_mo
    return;
  }
  let text = `⚽ *المباريات القادمة* (${upcomingMatches.length})*\n\n`;
  for (const m of upcomingMatches.slice(0,15)) {
    const city = CITIES.find(c => c.id === m.cityId);
    const dist = city ? `${city.distFromHerne} كم` : "?";
    const time = new Date(m.time).toLocaleString("de-DE", { weekday:"short", day:
    text += `*${time}*\n${m.home} ضد ${m.away}\n🏟️ ${m.venue} (${dist})\n_${m.com
  }
  await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
});

```

```

bot.onText(/\\/events/, async (msg) => {
  await bot.sendMessage(msg.chat.id, "🕒 Loading events...");
  upcomingEvents = await fetchUpcomingEvents();
  if (!upcomingEvents.length) {
    await bot.sendMessage(msg.chat.id, "لا أحداث قادمة", { parse_mode:"Markdown

```



```

    return;
}
let text = `🎵 *الأحداث القادمة* (${upcomingEvents.length}):*\n\n`;
for (const ev of upcomingEvents.slice(0,15)) {
    const city = CITIES.find(c => c.id === ev.cityId);
    const dist = city ? `${city.distFromHerne} كم` : "?";
    const time = new Date(ev.time).toLocaleString("de-DE", { weekday:"short", day
    text += `*${time}*\n${ev.name}\n🏠 ${ev.venue} (${dist})\n\n`;
}
await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
});

bot.onText(/\//strikes/, async (msg) => {
    await bot.sendMessage(msg.chat.id, "🔍 Checking strikes...");
    const strikes = await fetchStrikeNews();
    const dis = await fetchTransportDisruptions();
    let text = "🇸🇩 *الإضرابات والتعطلات:*\n\n";
    if (!strikes.length && !dis.length) {
        text += "✅ لا تعطلات حالياً";
    } else {
        if (strikes.length) {
            text += "⚠️ *إضرابات:*\n";
            for (const s of strikes) text += `• ${s.title}\n`;
            text += "\n";
        }
        if (dis.length) {
            text += "🚆 *تعطلات قطارات:*\n";
            for (const d of dis) {
                const city = CITIES.find(c => c.id === d.cityId);
                text += `• ${d.station} - ${d.cancelled} ملغية، ${d.delayed} متأخرة\n`;
            }
        }
    }
    await bot.sendMessage(msg.chat.id, text, { parse_mode:"Markdown" });
});

bot.onText(/\//crowds(?:\s+(\w+))?/, async (msg, match) => {
    const arg = match[1]?.toLowerCase();
    if (!arg) { await bot.sendMessage(msg.chat.id, "Usage: `/crowds bochum` or `/cr
    if (arg === "all") {
        await bot.sendMessage(msg.chat.id, `🔍 Scraping (~${(CITIES.length*0.02).toFi
        for (const city of CITIES) {
            try { await bot.sendMessage(msg.chat.id, `🔍 ${city.name}...`); await scrape
            catch(e) { await bot.sendMessage(msg.chat.id, `❌ ${city.name}: ${e.message
        }
        return;
    }
}

```

```

const city = CITIES.find(c => c.id === arg);
if (!city) { await bot.sendMessage(msg.chat.id, `❌ Unknown: ${arg}`); return;
await bot.sendMessage(msg.chat.id, `❏ Scraping *${city.name}*...`, { parse_mode
try { await scrapeOneCity(city, msg.chat.id); }
catch(e) { await bot.sendMessage(msg.chat.id, `❌ ${e.message}`); }
});

```

```

// ——— START —————
console.log("NRW Surge Bot v7 started.");
bot.sendMessage(CHAT_ID,
`✅ *NRW Surge Bot v7*\n${CITIES.length} مدينة · ${AIRPORTS.length} مطارات\n\n
{ parse_mode:"Markdown" });

```

```

// Smart traffic scan – runs every hour during peak hours only
setInterval(trafficScan, 60 * 60 * 1000);
// Flight scan every 2 hours
setInterval(flightScan, 2 * 60 * 60 * 1000);
// Match alerts every 5 min
setInterval(matchAlertScan, 5 * 60 * 1000);
// Event alerts every 10 min
setInterval(eventAlertScan, 10 * 60 * 1000);
// Disruption scan every 15 min
setInterval(disruptionScan, 15 * 60 * 1000);
// Daily report scheduler
scheduleDailyReport();

```

```

// Initial runs
trafficScan();
flightScan();
matchAlertScan();
eventAlertScan();
disruptionScan();

```